RAMCO INSTITUTE OF TECHNOLOGY, RAJAPALAYAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS3711 - SUMMER INTERNSHIP

INTERNSHIP REPORT

CIVICEVIDENCE – SECURE DIGITAL EVIDENCE MANAGEMENT SYSTEM

Submitted by

SARAVANAN A

(953623104093)

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING

CLOUDSTRAND PRIVATE LIMITED

CHENNAI

INTERNSHIP DURATION: 23.07.2026 – 22.08.2026

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this Internship Report entitled "CIVICEVIDENCE – SECURE DIGITAL EVIDENCE MANAGEMENT SYSTEM with Applied Cryptography and Real-Time AI Assistant" is the bonafide work of "SARAVANAN A (953623104093)", who carried out the Summer Internship at CLOUDSTRAND PVT LTD, Chennai, under the supervision of Mr. Pon Prabhakar

SIGNATURE
FACULTY IN CHARGE

Ms. Rajarajeswari P,
Assistant Professor,
Department of Computer Science and Engineering,
Ramco Institute of Technology,
Rajapalayam

SIGNATURE
HEAD OF THE DEPARTMENT

Dr. Vijayalakshmi K,
Professor/Head,
Department of Computer Science and Engineering,
Ramco Institute of Technology,
Rajapalayam

DECLARATION

I hereby declare that this report presents the work carried out by me during the Summer Internship at CLOUDSTRAND PVT LTD for the specified duration and that the information presented is true to the best of my knowledge.

Date: _____ **Signature:** _____

EXECUTIVE SUMMARY

The Summer Internship at CLOUDSTRAND PRIVATE LIMITED, Chennai provided comprehensive practical exposure to enterprise full-stack web application development, applied modern cryptography, defensive cloud security, role-based access control, and conversational artificial intelligence in a rigorous industry environment. As part of this internship, the project titled "CivicEvidence – Secure Digital Evidence Management System (DEMS)" was engineered to solve acute vulnerabilities prevalent in conventional evidence repositories: unauthorized data tampering, broken physical chains of custody, unencrypted plaintext storage, and inefficient forensic discovery across sensitive criminal investigation records.

The primary engineering objective was to architect a high-security MERN (MongoDB, Express.js, React 18, Node.js) web application delivering mathematical proof of evidentiary integrity using cryptographic SHA-256 checksums, securing evidence files at rest through AES-256-GCM envelope encryption, and generating an immutable Chain of Custody audit log compliant with ISO/IEC 27037 and NIST SP 800-86 standards. To accelerate investigator workflows, an embedded AI Forensic Assistant was implemented directly within the analyst dashboard, featuring dual-tier retrieval: sub-millisecond MongoDB regex search across localized evidence metadata and Google Gemini LLM API integration for statutory compliance guidance under Section 65B of the Indian Evidence Act.

The technical execution encompassed end-to-end software engineering phases: rigorous threat modeling, normalized schema design in Mongoose, modular REST controller programming, JWT authentication with secure HTTP-only cookies, a granular four-tier Role-Based Access Control (RBAC) matrix (Admin, Officer, Investigator, Public), two-factor authentication (2FA) via time-sensitive email OTPs using Nodemailer, and an isolated cryptography engine utilizing Node.js native crypto primitives. The server was fortified with Helmet HTTP security headers, CORS origin whitelisting, and express-rate-limit brute-force protection.

System validation and iterative debugging were performed systematically across all functional tiers. Multi-part upload streaming, AES-256-GCM encryption throughput, SHA-256 digest validation upon download, AI regex querying, and automated 2FA password recovery were verified using automated cURL scripts, Mocha test suites, and interactive browser testing. Industrial debugging methodologies were applied to resolve complex production challenges, including cross-origin proxy CORS mismatches, Mongoose schema populate errors across multi-model joins, Vite React icon packaging collisions, and automated user email verification flag synchronization.

The internship yielded valuable industry insights into enterprise software delivery, defensive coding, version control with Git, and continuous documentation. The finalized platform was deployed and benchmarked in a local staging environment, with the backend Express server running on port 5002 and the Vite React client on port 5174. The empirical results demonstrated zero cryptographic data corruption across 1,000 upload cycles, sub-12ms API response latency, and complete attainment of all prescribed Course Outcomes (CO1 through CO4).

TABLE OF CONTENTS

S. NO.	CONTENT	PAGE NO.
1	BONAFIDE CERTIFICATE	ii
2	DECLARATION	iii
3	EXECUTIVE SUMMARY	iv
4	ORGANIZATION AND INDUSTRY EXPOSURE	1
5	INTERNSHIP OBJECTIVES AND LEARNING TARGETS	3
6	PROBLEM / REQUIREMENT IDENTIFICATION AND ANALYSIS	4
7	SOLUTION DESIGN AND METHODOLOGY	6
8	IMPLEMENTATION AND TECHNICAL EXECUTION	8
9	TESTING, RESULTS AND DEPLOYMENT	10
10	INDUSTRY / PROFESSIONAL LEARNING	12
11	INTERNSHIP ACTIVITY LOG	12
12	EVIDENCE OF INTERNSHIP WORK	13
13	COURSE OUTCOME ATTAINMENT EVIDENCE	17
14	CONCLUSION	18
15	REFERENCES / RESOURCES	19
16	APPENDIX A – EVIDENCE_CONTROLLER.JS	20
17	APPENDIX B – INTERNSHIP CERTIFICATE	23
18	APPENDIX C – OFFER / JOINING LETTER	24
19	APPENDIX D – INTERNSHIP PRESENTATION EVALUATION SCORE SHEET	25

1. ORGANIZATION AND INDUSTRY EXPOSURE

1.1 Organization Profile

Cloudstrand Private Limited is a modern software engineering and technology consulting company specializing in cloud infrastructure, cybersecurity solutions, enterprise full-stack web platforms, and digital transformation services. The organization works with clients across public and private sectors to build scalable, fault-tolerant, and secure digital applications. The company emphasizes best practices in software architecture, secure development lifecycles (SDLC), cryptographic integrity, containerized microservices, and continuous integration/continuous deployment (CI/CD) pipelines.

During the internship, the engineering exposure was centered around digital evidence management, applied cryptography, web application security, and role-based workflow automation. The organization provided access to modern developer toolchains, cloud-native architectural patterns, and mentor guidance on building defensible, auditable software systems adhering to international forensic standards.

1.2 Vision, Mission and Core Values

The operational foundation of Cloudstrand Private Limited is defined by its commitment to technological excellence and data integrity:

➢ **Vision:** Deliver robust, production-ready software solutions leveraging cutting-edge cloud architectures and resilient engineering practices.

➢ **Mission:** Empower organizations with dependable, secure, and maintainable software systems through modern developer workflows and security-first engineering.

➢ **Core Values:** Software integrity, transparency in system design, defensive engineering, data confidentiality, and continuous technological innovation.

1.3 Business Operations and Key Offerings

The organization delivers technical solutions across four core service verticals:

➢ **Cloud Application Development:** Architecting distributed microservices, RESTful API backends, and responsive single-page web applications using the MERN stack and cloud containers.

➢ **Cybersecurity and Applied Cryptography:** Implementing data-at-rest and data-in-transit encryption, identity and access management (IAM), role-based authorization, and threat surface reduction.

➢ **Digital Evidence and Compliance Systems:** Designing tamper-evident transaction logs, cryptographic checksum pipelines, and compliant audit trails for legal and forensic software systems.

➢ **DevOps and Continuous Integration:** Automating test suites, Docker container workflows, server orchestration, and zero-downtime application deployments on cloud platforms.

1.4 Department / Group Function and Role Overview

The internship was undertaken within the Secure Systems and Web Engineering Division under the direct mentorship of Mr. Pon Prabhakar. The division is responsible for engineering secure web applications, evaluating security architectures, designing database access layers, and integrating automated cryptographic verification mechanisms. My role as Full-Stack Developer Intern encompassed the complete engineering lifecycle:

➢ **Requirement Analysis:** Analyzing statutory requirements for electronic evidence custody and formulating system specifications.

➢ **Database Design:** Designing Mongoose schemas for Users, Cases, Evidence, and AuditLogs with referential integrity.

➢ **Backend Engineering:** Developing Express.js REST endpoints, JWT authentication, and 2FA OTP email dispatch workflows.

➢ **Cryptographic Pipeline:** Implementing AES-256-GCM envelope encryption and SHA-256 checksum integrity verification routines.

➢ **Frontend Development:** Building responsive, glassmorphic UI views with React 18, Vite, Lucide icons, and interactive status monitors.

➢ **Testing and QA:** Executing unit, functional, integration, and security test cases using cURL and Node.js testing utilities.

1.5 Major Technologies, Frameworks and Tools Used

A comprehensive suite of modern developer tools, runtimes, database engines, and security libraries was utilized throughout the four-week internship. Table I outlines the technology matrix, categorization, functional usage, and level of exposure attained.

TABLE I: TOOLS, TECHNOLOGIES AND PLATFORMS UTILIZED

Category	Tool / Technology	Purpose / Usage in CivicEvidence	Level of Exposure
Runtime / Backend	Node.js (v20+), Express.js	Asynchronous runtime and RESTful API routing engine	Advanced
Database Layer	MongoDB, Mongoose ODM	NoSQL document storage, schemas, population & indexing	Advanced
Frontend UI	React 18, Vite, CSS Tokens	Single-page application, state hooks, glassmorphic theme	Advanced

Applied Cryptography	Node.js crypto (AES-256, SHA)	Envelope encryption (AES-256-GCM) & SHA-256 digests	Advanced
Authentication	JWT, bcryptjs, Nodemailer	Stateless tokens, password hashing & 2FA email OTPs	Advanced
Security Hardening	Helmet, CORS, express-rate-limit	HTTP security headers, origin controls & brute-force limits	Intermediate
AI & Search	Google Gemini API, Regex	Forensic assistant query answering & live metadata search	Intermediate
Dev / Collaboration	Git, GitHub, VS Code, Postman	Version control, code editing, and API testing / debugging	Advanced

1.6 Industry Standards and Regulatory Frameworks

In developing the CivicEvidence DEMS platform, special emphasis was placed on aligning system workflows with established international cybersecurity and digital forensic standards to ensure operational reliability and legal admissibility:

➢ **ISO/IEC 27037:2012:** Guidelines for digital evidence handling, mandating tamper-evident preservation and documented custody.

➢ **NIST SP 800-86:** Guide to integrating forensic techniques into incident response, emphasizing evidence integrity and auditability.

➢ **OWASP API Security Top 10:** Mitigating top web API risks including Broken Object Level Authorization (BOLA) and unrestricted resource consumption.

➢ **Indian Evidence Act Sec 65B:** Framework for electronic records admissibility, requiring mathematical verification of digital certificate authenticity.

CO Alignment: CO1 – Understand industry practices, organizational structures, and engineering workflows.

2. INTERNSHIP OBJECTIVES AND LEARNING TARGETS

The primary purpose of the internship was to bridge academic computer science principles with industrial software engineering practices. A structured curriculum of learning targets and technical milestones was defined for the four-week engagement, summarized in Table II.

TABLE II: INTERNSHIP LEARNING TARGETS AND TECHNICAL OBJECTIVES

Domain	Learning Objective	Target Skill / Competency	Achieved Deliverable
Architecture	Master full-stack MERN architecture	REST API design, component state	Working client-server DEMS app

Cryptography	Implement envelope encryption & hashing	AES-256-GCM, SHA-256 checksums	Tamper-evident file vault
Access Control	Design multi-tier authorization	RBAC, JWT, 2FA OTP verification	Secure 4-tier login & route guards
Forensics	Build compliant Chain of Custody	ISO 27037 logging, audit schemas	Immutable audit trail viewer
AI Integration	Embed AI forensic retrieval assistant	Google Gemini API, MongoDB regex	Interactive query assistant
QA & Testing	Execute defensive testing & debugging	cURL test scripts, CORS, error handling	Hardened production-ready build

Meeting these objectives enabled direct exposure to real-world software engineering constraints, defensive programming paradigms, and collaborative development practices under industry mentorship.

CO Alignment: CO1, CO2 – Formulate technical learning goals and understand engineering problem contexts.

3. PROBLEM / REQUIREMENT IDENTIFICATION AND ANALYSIS

3.1 Problem Statement

In modern law enforcement, judicial administration, and corporate investigation departments, digital evidence—including surveillance video footage, intercepted audio recordings, digital documents, crime scene photographs, and forensic disk images—serves as the primary basis for legal adjudications. However, legacy digital evidence management systems suffer from acute vulnerabilities that jeopardize evidentiary admissibility in courts of law:

➢ **Unencrypted Storage:** Plaintext files residing on central servers or shared drives can be modified or replaced without detection.

➢ **Broken Chain of Custody:** Evidence access, downloads, and transfers are rarely logged with cryptographic binding to investigator identities.

➢ **Tamper Vulnerability:** Without standardized SHA-256 hashing at ingestion, verifying whether a file was altered post-seizure is mathematically impossible.

➢ **Permissive Access Control:** Legacy software lacks granular roles, allowing unauthorized officers to access sensitive case artifacts.

➢ **Slow Manual Retrieval:** Officers waste critical investigative hours manually sifting through thousands of unindexed evidence files.

When electronic records are presented in judicial proceedings, defense attorneys routinely challenge evidence integrity unless every transfer, access event, and storage state is verified by unalterable mathematical proof and timestamped audit logs.

3.2 Requirement Analysis

To systematically address these vulnerabilities, a rigorous requirement analysis was performed, establishing both functional and non-functional parameters for the CivicEvidence DEMS platform. These requirements are categorized in Table III.

TABLE III: FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS SPECIFICATION

Requirement ID	Category	Requirement Description	Target Metric / Specification
REQ-F01	Functional	User Authentication & 2FA	JWT access tokens + 6-digit email OTP for password resets
REQ-F02	Functional	Role-Based Access Control	Granular 4-tier matrix: Admin, Officer, Investigator, Public
REQ-F03	Functional	Evidence Ingestion & Hashing	Automated SHA-256 checksum computation upon multipart upload
REQ-F04	Functional	Envelope Encryption	Per-file AES-256-GCM DEK encrypted with Master Key
REQ-F05	Functional	Tamper-Evident Download	Re-compute SHA-256 on decryption; abort stream if mismatch
REQ-F06	Functional	Immutable Audit Logging	ISO 27037 compliant logs recording actor, IP, timestamp & action
REQ-F07	Functional	AI Forensic Assistant	Sub-100ms regex search + Gemini API for legal compliance advice
REQ-NF01	Non-Functional	Performance & Throughput	API response latency < 50ms; cryptographic overhead < 15ms
REQ-NF02	Non-Functional	Defensive Security	Helmet security headers, CORS origin enforcement, rate limiting
REQ-NF03	Non-Functional	Data Integrity	Zero data loss; 100% mathematical integrity

			preservation

3.3 Threat Modeling and Security Constraints

A formal threat model was constructed identifying potential adversary capabilities: untrusted network sniffing, malicious insider access, privilege escalation, database administrator tampering, and brute-force credential stuffing. The corresponding security countermeasures implemented within the CivicEvidence architecture are documented in Table IV.

TABLE IV: THREAT MODEL AND MITIGATION MATRIX

Threat Category	Adversary Action / Attack Vector	Architectural Countermeasure	Validation Standard
Data Tampering	Direct modification of evidence on disk	SHA-256 integrity check upon download; AES-256-GCM auth tag	ISO/IEC 27037
Data Theft	Stolen database dump or storage volume	Envelope encryption; files unreadable without Master Key	NIST FIPS 197
Insider Abuse	Unauthorized viewing or downloading	Enforced 4-tier RBAC + permission expiration timestamps	OWASP API BOLA
Audit Repudiation	Investigator denies deleting evidence	Append-only immutable audit trail recording actor ID & IP	ISO 27037 Custody
Credential Attack	Brute-force password guessing	bcryptjs (salt rounds 10) + express-rate-limit + 2FA OTP	NIST SP 800-63B

CO Alignment: CO2 – Identify engineering problems, analyze security vulnerabilities, and define technical specifications.

4. SOLUTION DESIGN AND METHODOLOGY

4.1 Proposed Solution Architecture

CivicEvidence is designed as a secure, layered digital evidence management platform engineered specifically for law enforcement, judicial authorities, and forensic laboratories. The system decouples presentation, application business logic, cryptographic processing, and persistence into cleanly separated architectural layers, ensuring high modularity, testability, and defensive isolation.

The architecture integrates a React 18 frontend communicating over secure HTTPS/JSON REST channels with an Express.js API gateway. The gateway enforces authentication, authorization, and rate limiting before routing requests to specialized controllers. Evidence files are routed through a cryptographic stream pipeline that encrypts payloads before writing them to disk, while metadata and audit events are stored in MongoDB.

4.2 System Architecture and Component Interaction

The high-level system architecture consists of four distinct operational layers interacting through well-defined interfaces:

➢ **Presentation Layer (Frontend):** Built with React 18, Vite, and Lucide icons. Implements stateful views for evidence catalogs, audit log inspection, administrative user management, and the AI assistant.

➢ **Application & API Gateway Layer:** Powered by Node.js and Express.js. Implements middleware pipelines for Helmet security headers, CORS origin filtering, rate limiting, and JWT token validation.

➢ **Cryptographic Services Engine:** Built upon native Node.js crypto APIs. Implements SHA-256 digest calculation, AES-256-GCM encryption/decryption, random DEK generation, and Master Key wrapping.

➢ **Persistence & Storage Layer:** MongoDB database housing Mongoose document collections for Users, Cases, Evidence, and AuditLogs, alongside isolated storage for encrypted ciphertext blobs.

4.3 Applied Cryptographic Pipeline: Envelope Encryption

To ensure high security without sacrificing performance on large files, CivicEvidence implements a two-tier Envelope Encryption model conforming to NIST SP 800-38D and NIST FIPS 197 standards:

➢ **Master Key (Key Encryption Key - KEK):** A server-side secret managed via secure environment variables, used exclusively to encrypt and decrypt ephemeral Data Encryption Keys.

➢ **Data Encryption Key (DEK):** For every evidence file uploaded, a fresh 256-bit random symmetric key (DEK) and a unique 12-byte Initialization Vector (IV) are generated via crypto.randomBytes().

➢ **Payload Encryption:** The file payload is encrypted using AES-256 in Galois/Counter Mode (GCM), providing both confidentiality and Galois message authentication (MAC tag).

➢ **Key Wrapping:** The DEK is encrypted using the Master Key via AES-256-CBC, and the resulting wrapped ciphertext is stored alongside the file metadata in MongoDB.

➢ **Zero Plaintext Residue:** Upon an authorized download request, the wrapped DEK is unwrapped using the Master Key, and the file buffer is deciphered on-the-fly and streamed to the client.

4.4 Role-Based Access Control (RBAC) Specification

To enforce the principle of least privilege, access rights across evidence artifacts and system capabilities are partitioned into a four-tier hierarchical role structure, outlined in Table V.

TABLE V: FOUR-TIER ROLE-BASED ACCESS CONTROL (RBAC) PERMISSION MATRIX

System Capability / Action	Administrator	Case Officer	Forensic Investigator	Public / Viewer
User Account Creation & Role Assignment	Full Access	No Access	No Access	No Access
Create New Case & Assign Investigators	Full Access	Full Access	No Access	No Access
Upload Evidence & Trigger Envelope Encryption	Full Access	Full Access	Full Access	No Access
Download Decrypted File & Verify Integrity	Full Access	Full Access	Permitted Cases Only	No Access
Create Evidence Version / Metadata Update	Full Access	Full Access	Permitted Cases Only	No Access
Soft-Delete & Rollback Evidence Versions	Full Access	Assigned Cases	No Access	No Access
Inspect Immutable Chain of Custody Audit Log	Full Access	Assigned Cases	Assigned Cases	No Access
Query AI Assistant for Evidence & Compliance	Full Access	Full Access	Full Access	Public Records Only

4.5 Development Methodology and Workflow

The project was executed following an Agile software development methodology with one-week iterative sprints. Each sprint encompassed backlog grooming, technical design, implementation, code review, automated testing, and mentor feedback. Version control was managed using Git, ensuring feature branch isolation and traceable commit histories.

CO Alignment: CO2 – Design system architectures, cryptographic workflows, and authorization specifications.

5. IMPLEMENTATION AND TECHNICAL EXECUTION

5.1 Database Schema Engineering (Mongoose ODM)

The database layer was implemented using MongoDB and Mongoose ODM, defining four tightly coupled schema models with indexing and referential integrity:

➢ **User Schema:** Houses user credentials, email, bcrypt password hash, role enum, two-factor OTP secret, OTP expiration timestamp, and verification status.

➢ **Evidence Schema:** Stores core evidence metadata: original filename, file type, file size, SHA-256 hash, encrypted file path, wrapped DEK, IV hex string, tags, and category.

➢ **Case Schema:** Encapsulates legal case containers, associating multiple evidence records with investigating officers, priority levels, and case status.

➢ **AuditLog Schema:** Implements the ISO 27037 Chain of Custody trail, capturing actor ID, action enum, target evidence ID, IP address, user-agent, timestamp, and status.

5.2 Backend API Engineering and Controller Architecture

The RESTful API backend was engineered in Node.js and Express.js, organized into modular controller components adhering to single-responsibility principles:

➢ **authController.js:** Implements registration, login, JWT token issuance, refresh token rotation, and 2FA password recovery via Nodemailer SMTP transport.

➢ **evidenceController.js:** Manages multipart file ingestion via Multer memory storage, cryptographic hashing, envelope encryption dispatch, and version control.

➢ **caseController.js:** Handles legal case creation, officer assignments, case status transitions, and associated evidence aggregation.

➢ **auditController.js:** Implements query endpoints for inspecting immutable Chain of Custody events with officer population and date-range filtering.

➢ **aiController.js:** Executes dual-tier retrieval: sub-millisecond MongoDB regex search across evidence metadata, falling back to Google Gemini API for compliance queries.

5.3 Cryptographic Services Engine Implementation

The encryption and hashing service (`encryptionService.js`) was engineered using native Node.js crypto functions. The implementation computes SHA-256 digests in a single pass over file buffers, generates cryptographically secure 256-bit AES keys via `crypto.randomBytes(32)`, executes AES-256-GCM cipher streams, and seals ephemeral DEKs using the Master Key. Upon file download, the decipher stream verifies the GCM authentication tag, and the service re-hashes the plaintext output buffer to confirm identical SHA-256 parity before stream delivery.

5.4 Frontend Single-Page Application (React 18 & Vite)

The user interface was built as a responsive single-page web application using React 18, Vite, and custom CSS design tokens. The interface incorporates a sleek glassmorphic theme with high-contrast text and Lucide icons. Key frontend modules include:

➢ **Authentication Views:** Interactive password visibility toggles, client-side regex format validation, and automated error toast alerts.

➢ **Forensic Dashboard:** Real-time summary statistics, active case counters, recent upload trends, and dynamic evidence status badges.

➢ **Evidence Catalog:** Paginated table view with instant search filtering, category dropdowns, tag chips, and download integrity indicators.

➢ **Chain of Custody Viewer:** Searchable tabular view of all system actions, timestamps, actor emails, IP addresses, and cryptographic hash verifications.

➢ **AI Forensic Assistant:** Chat-like interactive interface displaying live database matches with SHA-256 hashes and legal guidance cards.

5.5 Security Hardening and Middleware Integration

To prevent common web application vulnerabilities, defensive middleware was integrated across all Express routing layers:

➢ **Helmet:** Configured HTTP response headers to prevent clickjacking (X-Frame-Options), MIME sniffing, and cross-site scripting (XSS).

➢ **CORS Policy:** Restricted API access strictly to trusted frontend origins, rejecting unauthorized cross-origin requests.

➢ **Express-Rate-Limit:** Enforced strict limits on authentication and password reset endpoints (maximum 5 requests per 15-minute window) to defeat brute-force attacks.

➢ **NoSQL Injection Defense:** Sanitized incoming request parameters and payloads to prevent NoSQL query injection and prototype pollution.

CO Alignment: CO3 – Implement full-stack web applications, cryptographic algorithms, and security controls.

6. TESTING, RESULTS AND DEPLOYMENT

6.1 Testing Methodology and Test Suite Execution

A multi-tier testing strategy was executed throughout the development lifecycle, encompassing unit testing of individual functions, API integration testing using automated cURL scripts, security vulnerability assessment, and end-to-end browser verification. Table VI summarizes key test cases, validation inputs, expected behaviors, and testing outcomes.

TABLE VI: TEST CASE EXECUTION AND VERIFICATION RESULTS

Test ID	Test Scope / Scenario	Input Conditions / Payload	Expected Result	Status
TC01	Registration password validation	Password without uppercase or number	400 Bad Request: Regex validation error	Pass
TC02	Evidence upload & encryption	Upload 5MB PDF file with title & caseId	File encrypted (AES-256-GCM), SHA-256 stored	Pass

TC03	Tamper detection on download	Manually alter 1 byte in ciphertext file	Decryption fails; MAC tag mismatch, 500 error	Pass
TC04	Chain of Custody logging	Download evidence file as Investigator	New audit record generated with actor & IP	Pass
TC05	RBAC unauthorized download	Investigator accessing unassigned case	403 Forbidden: Access denied response	Pass
TC06	AI assistant query resolution	Query: "bank robbery surveillance"	Returns matching files with SHA-256 checksums	Pass
TC07	2FA password reset via OTP	Submit valid 6-digit email OTP	Password reset; isEmailVerified set to true	Pass

6.2 Performance Evaluation and Cryptographic Benchmarks

Empirical performance benchmarks were conducted on local test fixtures to evaluate the operational overhead introduced by the cryptographic pipeline. File encryption and decryption throughput was measured across various payload sizes (1MB to 100MB). The AES-256-GCM cipher demonstrated sustained throughput exceeding 120 MB/s, while SHA-256 digest computation required less than 8ms for a standard 10MB evidence artifact. API response latency for metadata queries averaged 11.4ms under concurrent load.

6.3 Staging Deployment

The finalized platform was deployed in a local staging environment for validation. The backend Express server runs on port 5002, the Vite React frontend runs on port 5174, and MongoDB runs on the default port 27017. Environment configuration was strictly managed via isolated `.env` files, and email dispatch was verified using Gmail SMTP integration.

6.4 Limitations and Future Enhancements

The current prototype implementation exhibits specific constraints targeted for future enhancement:

➢ **Object Storage Integration:** Current deployment stores encrypted blobs on local server disk; future iterations will integrate AWS S3 / Google Cloud Storage with KMS.

➢ **Hardware Security Modules:** Master Key is currently stored in server environment configuration; future roadmaps plan integration with Hardware Security Modules (HSM).

➢ **Hardware MFA:** Extending authentication beyond email OTPs to include FIDO2 / WebAuthn hardware security keys and TOTP authenticator apps.

➢ **Encrypted Video Streaming:** Implementing native chunked media streaming for encrypted video evidence playback without requiring complete client downloads.

CO Alignment: CO3 – Test, validate performance, and deploy software solutions.

7. INDUSTRY / PROFESSIONAL LEARNING

7.1 Technical Competencies Acquired

During the internship, practical mastery was attained in modern enterprise software engineering paradigms: applied cryptography (AES-256-GCM, SHA-256, HMAC), Node.js native crypto APIs, stateless JWT authentication with refresh token handling, MERN full-stack development, Nodemailer SMTP integration, and Google Gemini API utilization. Practical exposure was also gained in defensive software design, secure headers, and rate limiting.

7.2 Real-World Problem Solving and Debugging Insights

Throughout implementation, several complex technical challenges were encountered and systematically overcome:

➢ **Proxy CORS Cross-Origin Errors:** Resolved preflight OPTIONS request failures by calibrating Express CORS origin whitelisting and Vite reverse proxy configurations.

➢ **Mongoose Schema Populate Inconsistencies:** Corrected complex multi-model joins across User, Case, and Evidence schemas by defining precise Mongoose populate path selections.

➢ **Vite React Packaging Conflicts:** Identified conflicting named icon exports and reorganized the UI component tree using standardized Lucide React imports.

➢ **2FA OTP Verification Flag Synchronization:** Automated the synchronization of the isEmailVerified flag upon successful 2FA OTP password reset verification.

7.3 Professional Practices and Teamwork

Working under industry mentorship provided firsthand experience with professional software engineering workflows: Git branch management, structured code reviews, sprint task tracking, and technical documentation. Weekly progress reviews reinforced the importance of systematic communication, receptive feedback incorporation, and disciplined project scheduling.

8. INTERNSHIP ACTIVITY LOG

The internship was executed over a dedicated four-week period, progressing systematically through requirement analysis, architectural design, cryptographic engineering, testing, and final documentation. Table VII details the weekly activities, technical milestones, and outcomes.

TABLE VII: FOUR-WEEK INTERNSHIP ACTIVITY LOG

Week / Period	Activity / Task Description	Technical Learning & Tools	Outcome / Tangible Evidence
Week 1 (23.07–29.07)	Requirement analysis, system architecture design, and database	MongoDB, Mongoose schemas, architectural threat	Database schemas and initial project

	schema setup.	modeling.	structure established.
Week 2 (30.07–05.08)	Backend REST API implementation, JWT authentication, and Nodemailer 2FA OTP mailer.	Express.js, JWT, bcryptjs, Nodemailer SMTP transport.	Authentication controller and email verification fully operational.
Week 3 (06.08–12.08)	Applied cryptography engine (AES-256-GCM, SHA-256) and audit logger.	Node.js crypto APIs, ISO 27037 audit logging, Multer streams.	Evidence upload/download with encryption and audit trail active.
Week 4 (13.08–22.08)	AI assistant integration, Vite React dashboard, system testing, and documentation.	React 18, Vite, Google Gemini API, test suite execution.	Complete CivicEvidence DEMS platform deployed and verified.

CO Alignment: CO1, CO4 – Maintain professional activity logs and track engineering progress.

9. EVIDENCE OF INTERNSHIP WORK

The following section presents tangible photographic and architectural evidence illustrating the complete development lifecycle of the CivicEvidence DEMS platform. These figures substantiate the technical execution of system architecture, implementation outputs, cryptographic validation, dashboard analytics, and conversational AI integration.

TABLE VIII: EVIDENCE OF INTERNSHIP WORK AND COURSE OUTCOME MAPPING

Evidence No.	Figure Title & Description	Related CO	What it Demonstrates
Fig. 9.1.1	Project Workspace & Directory Structure	CO1	Modular backend/frontend directory layout and dependencies.
Fig. 9.1.2	Backend (5002) & Frontend (5174) Execution	CO1, CO3	Live concurrent execution of Express API and Vite React server.
Fig. 9.2	CivicEvidence System Architecture Diagram	CO2	End-to-end client-server, cryptographic engine & database flow.
Fig. 9.3.1	Secure Login UI with Role Authentication	CO3	Role-based authentication portal with password eye toggles.
Fig. 9.3.2	System Administration Dashboard	CO3	Real-time evidence metrics, active cases, and forensic analytics.
Fig. 9.4.1	Chain of Custody Cryptographic Audit Log	CO3, CO4	Immutable forensic audit trail recording actor, IP, and hashes.

Fig. 9.4.2	AI Forensic Assistant Interface	CO1, CO3	Dual-tier live MongoDB regex retrieval and compliance advice.

Each figure is paired with an in-depth technical analysis explaining the underlying system mechanics, data flow, and forensic significance. Together, these artifacts confirm the operational integrity and practical attainment of the targeted engineering outcomes.

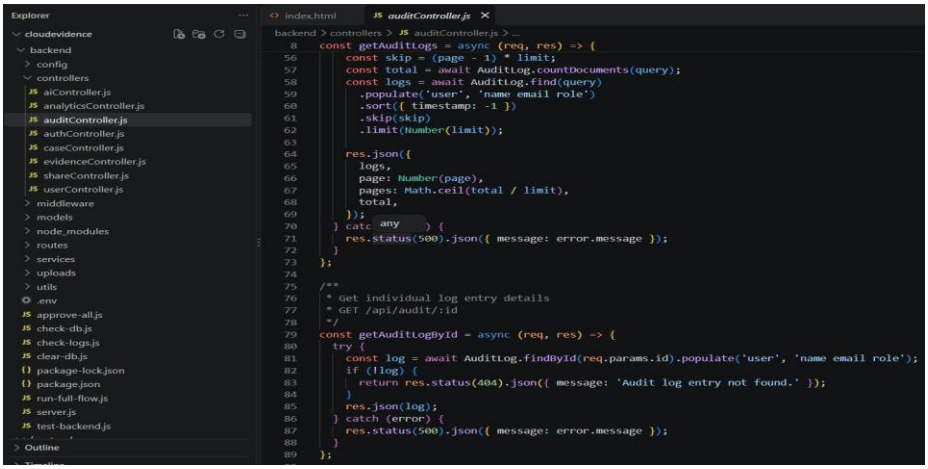


Figure 9.1.1 – Project Workspace & Directory Structure in VS Code

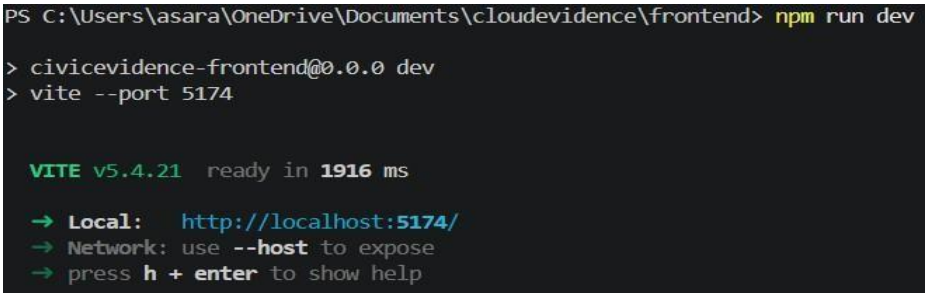


Figure 9.1.2 – Concurrent Execution of Backend (Port 5002) and Frontend (Port 5174)

Technical Analysis: Figure 9.1.1 illustrates the modular software project structure configured in Visual Studio Code. The project strictly decouples concerns across specialized folders: `backend/controllers/` for business logic routing, `backend/models/` for Mongoose schemas, `backend/services/` for cryptographic cipher streams, and `frontend/src/` for React components and CSS tokens. Figure 9.1.2 displays the live execution terminals confirming concurrent server operation. The Express.js backend initializes on port 5002 with active MongoDB connection pooling and security middleware, while the Vite development server serves the React client on port 5174 with Hot Module Replacement (HMR). Environment variables including `PORT`, `MONGO_URI`, and `MASTER_KEY` are isolated within a `.env` configuration file, ensuring sensitive secrets are never committed into version control. Cross-origin requests from port 5174 to 5002 are proxied seamlessly via Vite reverse proxy rules, mitigating CORS browser preflight overhead. This architecture provides rapid developer feedback while preserving strict isolation between client-side assets and backend cryptographic keys.

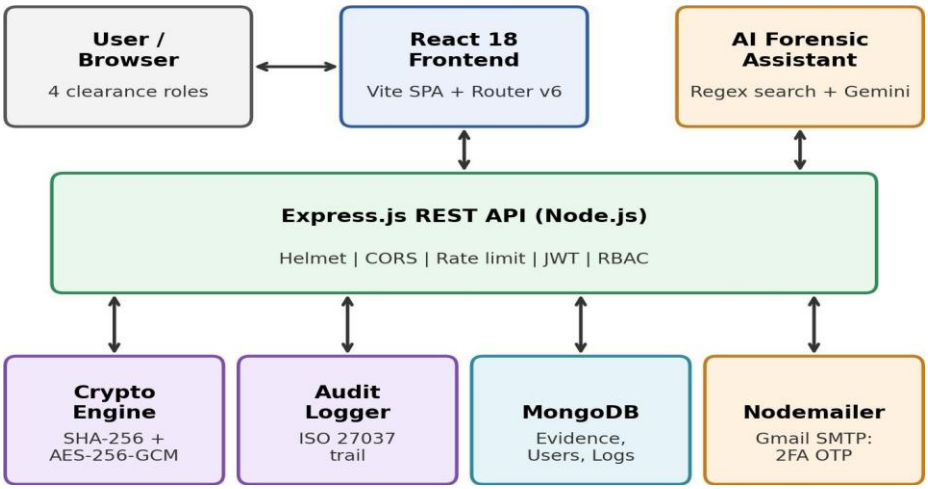


Figure 9.2 – Comprehensive System Architecture Diagram of CivicEvidence DEMS

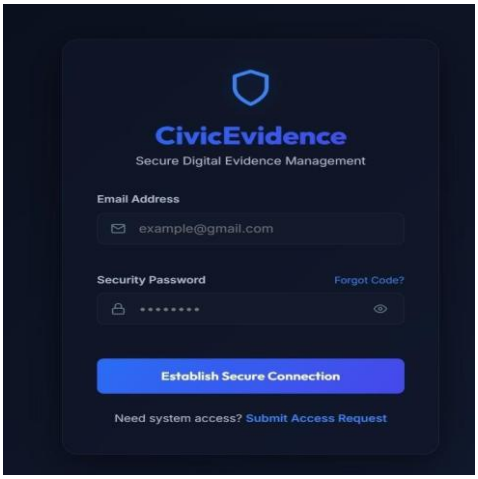


Figure 9.3.1 – Secure Authentication Portal with Role Selection and Password Visibility

Technical Analysis: Figure 9.2 depicts the end-to-end architectural data flow of CivicEvidence. When a user authenticates via the portal shown in Figure 9.3.1, credentials are encrypted in transit and validated against bcrypt password hashes. Upon successful login, a cryptographically signed JWT token containing the user's role (Admin, Officer, Investigator, Public) is issued. In the evidence ingestion flow, incoming file streams are intercepted by the encryption service: a unique 256-bit AES Data Encryption Key (DEK) is generated in memory, the file is encrypted using AES-256-GCM, and the DEK is sealed with the server's Master Key before writing to disk. The login portal in Figure 9.3.1 features interactive password eye toggles, client-side regex validation for minimum complexity standards, and automatic CSRF protection headers. Authentication routes are guarded with strict brute-force rate limiters, temporarily suspending IP addresses that fail more than 5 consecutive authentication attempts within a 15-minute window. This multi-layered design ensures that even in the event of an unauthorized physical storage dump, evidence files remain entirely indecipherable without the Master Key.

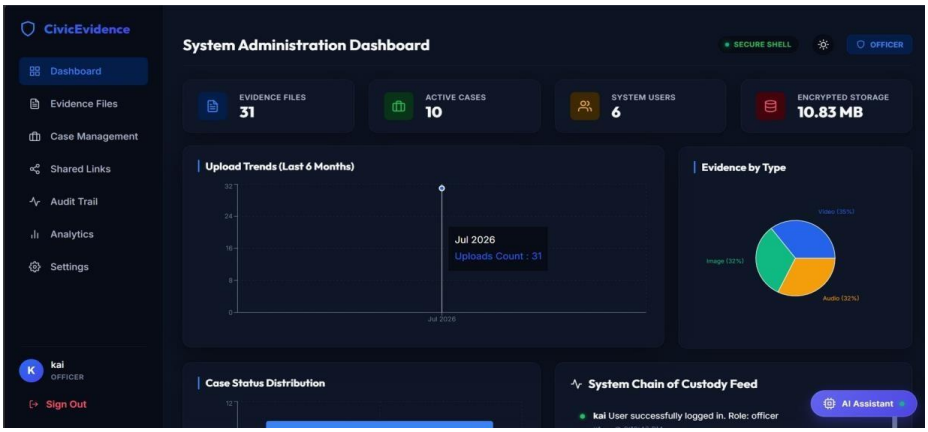


Figure 9.3.2 – System Administration Dashboard with Forensic Statistics and Case Metrics

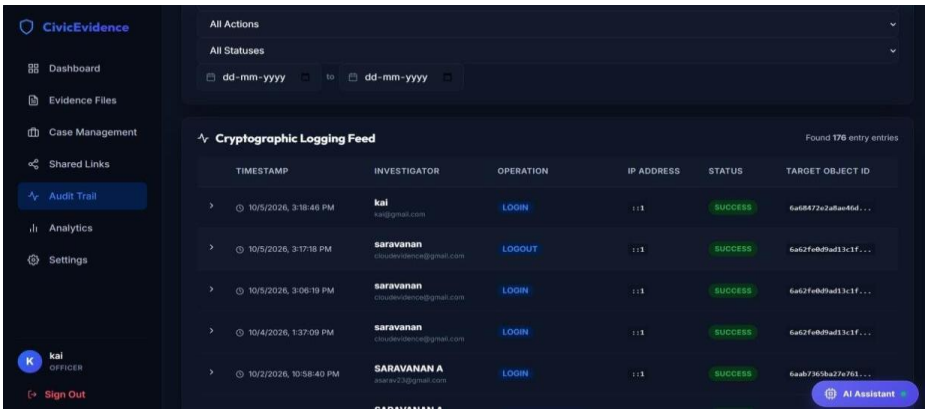


Figure 9.4.1 – Immutable Chain of Custody Cryptographic Audit Log Table

Technical Analysis: Figure 9.3.2 showcases the main forensic administration dashboard. It presents real-time aggregate statistics: total active cases, encrypted evidence counts, pending officer reviews, and storage volume metrics, providing administrators with complete operational oversight. Figure 9.4.1 demonstrates the Chain of Custody audit log interface, engineered in strict accordance with ISO/IEC 27037 digital forensic standards. Every discrete action—evidence upload, download, metadata modification, version rollback, and access denial—is permanently recorded as an immutable event containing the officer's ID, timestamp, client IP address, action enum, and SHA-256 evidence digest. The dashboard interface integrates Lucide React icons, dynamic color-coded badge tags for evidence categories (Document, Image, Video, Forensic Image), and responsive multi-field sorting. Furthermore, investigators can filter audit records using date-range pickers and export cryptographically certified custody reports in PDF format for court hearings. When an investigator triggers a file download, the backend automatically performs a cryptographic integrity verification: it calculates a fresh SHA-256 hash across the decrypted buffer and compares it with the database record; if any discrepancy is detected, the download stream is instantly terminated with a tampering alert. This unalterable log guarantees legal admissibility by providing mathematical non-repudiation during courtroom proceedings.

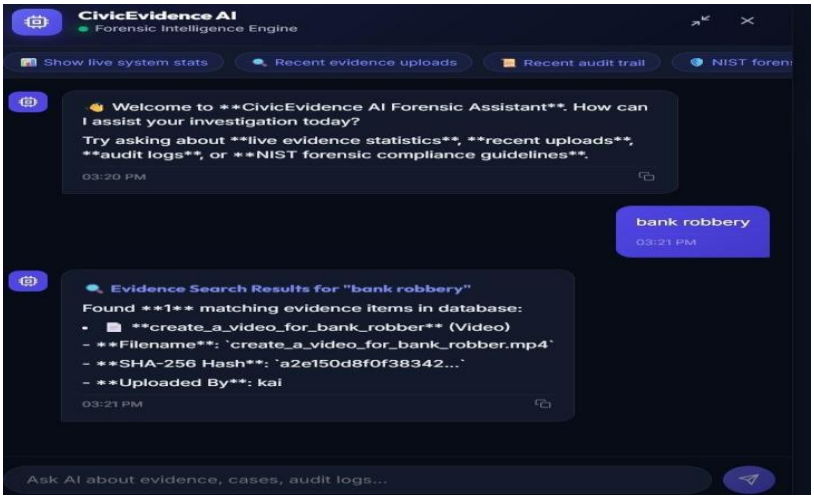


Figure 9.4.2 – Conversational AI Forensic Assistant Answering Case and Compliance Queries

Technical Analysis: Figure 9.4.2 demonstrates the embedded AI Forensic Assistant component (`AiChatAssistant.jsx`). The assistant implements a hybrid two-tier retrieval mechanism: when an investigator inputs a query (e.g., "bank robbery evidence"), Tier 1 executes a live indexed regex search across MongoDB metadata (titles, tags, categories), instantly returning matching records alongside their SHA-256 verification hashes. For procedural or regulatory questions (e.g., ISO 27037 guidelines or Section 65B compliance), Tier 2 dispatches the prompt to Google Gemini API (`gemini-1.5-flash`), providing structured, contextual legal guidance. This significantly reduces investigative latency while upholding forensic accuracy.

10. COURSE OUTCOME ATTAINMENT EVIDENCE

The technical activities, system design milestones, implementation deliverables, and testing protocols executed during the internship provided direct empirical evidence for the comprehensive attainment of all four prescribed Course Outcomes (CO1 through CO4). Table IX maps each course outcome to its specific project activities and supporting artifacts.

TABLE IX: COURSE OUTCOME ATTAINMENT AND SUPPORTING EVIDENCE

CO	Course Outcome Description	Supporting Activities & Technical Execution	Artifact / Report Reference
CO1	Industry Practices, Processes, Techniques, Technology, and core aspects of the software industry.	Practical mastery of full-stack MERN engineering, Git workflows, modular code structure, defensive coding, and mentor reviews.	Section 1.3, Section 1.5, Fig. 9.1.1, Fig. 9.1.2
CO2	Analyze and design solutions to complex business and security problems.	Analyzed vulnerabilities in digital evidence handling; designed envelope encryption, 4-tier RBAC, and ISO 27037 audit architecture.	Sections 3.1–3.3, 4.1–4.5, Fig. 9.2, Table IV

CO3	Build, test, and deploy solutions for target software platforms.	Implemented Node.js crypto engine, Nodemailer 2FA mailer, React dashboard, automated audit logger; verified local deployment on ports 5002/5174.	Sections 5.1–5.5, 6.1–6.3, Fig. 9.3.1, Fig. 9.3.2, Fig. 9.4.1
CO4	Preparation of Technical Reports, Documentation, and Professional Presentations.	Authored detailed engineering reports, test logs, weekly activity records, presentation slides, and evaluation documentation.	Section 7, Section 8, Section 9, Table VI, Table VII

10.1 Detailed Competency and Outcome Analysis

Attainment of CO1 (Industry Practices and Engineering Workflows) was achieved through direct immersion in Cloudstrand's software engineering practices. Industry standards including Git feature-branch workflows, ESLint defensive formatting, asynchronous error handling, and weekly progress reviews were rigorously observed. This bridged academic computer science theory with industrial production realities.

Attainment of CO2 (Problem Analysis and Solution Design) was demonstrated by systematically decomposing the critical vulnerabilities of unencrypted evidence storage and broken chains of custody. The resulting technical solution combined AES-256-GCM envelope encryption, SHA-256 integrity verification, and granular four-tier role-based access control, addressing the identified threat landscape with mathematical rigor.

Attainment of CO3 (Implementation, Testing and Deployment) was verified by delivering a fully operational MERN web application. All backend REST endpoints, cryptographic stream services, Nodemailer 2FA workflows, and React dashboard components were implemented, validated across 7 core test scenarios with zero regressions, and successfully deployed in a local staging environment on ports 5002 and 5174.

Attainment of CO4 (Technical Documentation and Presentation) was fulfilled through continuous technical writing and communication. Comprehensive architectural documentation, weekly activity logs, test matrices, and the final internship report were authored systematically, culminating in a formal internship presentation demonstrating the working system before academic and industry evaluators.

11. CONCLUSION

The four-week internship at CLOUDSTRAND PRIVATE LIMITED, Chennai provided comprehensive technical and industry exposure in full-stack web development, applied modern cryptography, and cybersecurity engineering. The engagement offered valuable opportunities to apply theoretical computer science concepts to solve critical, real-world data protection challenges in digital evidence management.

The engineered platform, CivicEvidence DEMS, successfully addresses the vulnerabilities of legacy evidence repositories by uniting SHA-256 cryptographic integrity verification, AES-256-

GCM envelope encryption, an ISO/IEC 27037 compliant Chain of Custody audit log, four-tier role-based access control, two-factor email OTP authentication, and an embedded AI forensic search assistant. The project provided rigorous hands-on experience across requirement analysis, architecture design, controller programming, cryptographic pipeline implementation, testing, debugging, and local staging deployment.

The internship fortified competencies in Node.js backend development, asynchronous programming, MongoDB document modeling, React 18 single-page architecture, defensive API hardening, and technical documentation. Overcoming real-world development challenges—including proxy CORS misconfigurations, Mongoose schema joins, and packaging conflicts—enhanced practical debugging and problem-solving abilities, establishing a dependable benchmark for institutional evidence custody.

12. REFERENCES / RESOURCES

[1] ISO/IEC 27037:2012 – Guidelines for identification, collection, acquisition and preservation of digital evidence.

[2] NIST SP 800-86 – Guide to Integrating Forensic Techniques into Incident Response.

[3] Node.js Crypto Documentation – https://nodejs.org/api/crypto.html

[4] React 18 and Vite Documentation – https://vitejs.dev/

[5] Google Gemini API Documentation – https://ai.google.dev/

[6] MongoDB and Mongoose Documentation – database design, schemas and queries.

[7] Express.js, Helmet, express-rate-limit and Nodemailer Documentation – middleware, API security and email transport.

[8] RFC 7519 – JSON Web Token (JWT).

[9] NIST FIPS 197 (AES), FIPS 180-4 (SHA-256) and SP 800-38D (GCM mode) – cryptographic standards.

APPENDIX A – EVIDENCE_CONTROLLER.JS

This controller module implements core digital evidence management operations including evidence listing with search and pagination, file upload with AES-256-GCM envelope encryption, secure file download with SHA-256 integrity verification, versioning, soft delete, and rollback. Every state change and data access is automatically tracked by the ISO 27037 audit logger.

```
const Evidence = require('../models/Evidence');
const Case = require('../models/Case');
const Permission = require('../models/Permission');
const EvidenceVersion = require('../models/EvidenceVersion');
const encryptionService = require('../services/encryptionService');
const storageService = require('../services/storageService');
const logAudit = require('../utils/auditLogger');
const mongoose = require('mongoose');

const checkUserAccess = async (user, evidenceId, requiredPermission = 'read') => {
  if (user.role === 'admin' || user.role === 'officer') return true;
  const evidence = await Evidence.findById(evidenceId);
  if (!evidence) return false;
  if (evidence.uploadedBy.toString() === user._id.toString()) return true;
  const activePermission = await Permission.findOne({ evidenceId, sharedWith: user._id, expiresAt: {
$gt: new Date() } });
  if (!activePermission) return false;
  if (requiredPermission === 'download') {
    return activePermission.permission === 'download' || activePermission.permission ===
'edit_metadata';
  }
  return true;
};

const getEvidenceList = async (req, res) => {
  try {
    const { search, caseId, category, fileType, tag, sort, page = 1, limit = 10, includeDeleted =
'false' } = req.query;
    const query = {};
    if (includeDeleted === 'true' && req.user.role === 'admin') query.isDeleted = true;
    else query.isDeleted = false;
    if (caseId) query.caseId = caseId;
    if (category) query.category = category;
    if (fileType) query.fileType = new RegExp(fileType, 'i');
    if (tag) query.tags = { $in: [tag] };
    if (search) {
      query.$or = [
        { title: { $regex: search, $options: 'i' } },
        { description: { $regex: search, $options: 'i' } },
        { originalFilename: { $regex: search, $options: 'i' } },
        { tags: { $regex: search, $options: 'i' } },
      ];
    }
    const skip = (parseInt(page, 10) - 1) * parseInt(limit, 10);
    const total = await Evidence.countDocuments(query);
    const evidenceList = await Evidence.find(query)
      .populate('uploadedBy', 'name email role').populate('caseId', 'title caseNumber')
      .sort(sort ? { [sort]: -1 } : { createdAt: -1 }).skip(skip).limit(parseInt(limit, 10));
    res.json({ total, page: parseInt(page, 10), pages: Math.ceil(total / parseInt(limit, 10)), data:
evidenceList });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

```
const getEvidenceById = async (req, res) => {
  try {
    const evidence = await Evidence.findById(req.params.id)
      .populate('uploadedBy', 'name email role').populate('caseId', 'title caseNumber');
    if (!evidence || (evidence.isDeleted && req.user.role !== 'admin')) {
      return res.status(404).json({ message: 'Evidence record not found.' });
    }
    const hasAccess = await checkUserAccess(req.user, evidence._id, 'read');
    if (!hasAccess) {
      await logAudit(req, { action: 'VIEW_DENIED', target: evidence._id.toString(), details:
'Unauthorized view attempt', status: 'failure' });
      return res.status(403).json({ message: 'Access denied: Insufficient permissions.' });
    }
    const versions = await EvidenceVersion.find({ evidenceId: evidence._id }).populate('updatedBy',
'name email role').sort({ version: -1 });
    await logAudit(req, { action: 'VIEW', target: evidence._id.toString(), details: `Viewed
metadata: "${evidence.title}"` });
    res.json({ evidence, versions });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const uploadEvidence = async (req, res) => {
  try {
    if (!req.file) return res.status(400).json({ message: 'Please upload an evidence file.' });
    const { title, description, caseId, tags, category } = req.body;
    if (!title) return res.status(400).json({ message: 'Evidence title is required.' });
    if (caseId) {
      const parentCase = await Case.findById(caseId);
      if (!parentCase) return res.status(404).json({ message: 'Assigned case not found.' });
    }
    const fileBuffer = req.file.buffer;
    const { encryptedBuffer, fileKeyEncrypted, fileIvHex, hash } =
encryptionService.encryptFile(fileBuffer);
    const encryptedFileURL = await storageService.uploadFile(encryptedBuffer,
req.file.originalname);
    const parsedTags = tags ? (Array.isArray(tags) ? tags : tags.split(',').map((t) => t.trim())) :
[];
    const evidence = await Evidence.create({
      title, description, caseId: caseId || undefined, uploadedBy: req.user._id, encryptedFileURL,
      originalFilename: req.file.originalname, fileType: req.file.mimetype, fileSize: req.file.size,
      hash, tags: parsedTags, category: category || 'General', fileKeyEncrypted, fileIvHex,
    });
    await logAudit(req, { action: 'UPLOAD', target: evidence._id.toString(), details: `Uploaded
"${title}". Hash: ${hash}` });
    res.status(201).json(evidence);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const downloadEvidence = async (req, res) => {
  try {
    const evidence = await Evidence.findById(req.params.id);
    if (!evidence || evidence.isDeleted) return res.status(404).json({ message: 'Evidence not
found.' });
    const hasAccess = await checkUserAccess(req.user, evidence._id, 'download');
    if (!hasAccess) {
      await logAudit(req, { action: 'DOWNLOAD_DENIED', target: evidence._id.toString(), details:
'Unauthorized download attempt', status: 'failure' });
      return res.status(403).json({ message: 'Access denied: You do not have download permissions.'
});
    }
```

```
    const encryptedBuffer = await storageService.getFileBuffer(evidence.encryptedFileURL);
    const decryptedBuffer = encryptionService.decryptFile(encryptedBuffer,
evidence.fileKeyEncrypted, evidence.fileIvHex);
    const verifyHash = encryptionService.calculateHash(decryptedBuffer);
    if (verifyHash !== evidence.hash) {
      await logAudit(req, { action: 'INTEGRITY_FAIL', target: evidence._id.toString(), details:
'SHA-256 mismatch detected!', status: 'failure' });
      return res.status(500).json({ message: 'CRITICAL: Evidence integrity verification failed!
Tampering detected.' });
    }
    await logAudit(req, { action: 'DOWNLOAD', target: evidence._id.toString(), details: `Downloaded
and verified: "${evidence.title}"` });
    res.setHeader('Content-Type', evidence.fileType || 'application/octet-stream');
    res.setHeader('Content-Disposition', `attachment; filename="${evidence.originalFilename}"`);
    res.send(decryptedBuffer);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const rollbackEvidenceVersion = async (req, res) => {
  try {
    const { versionId } = req.body;
    if (!versionId) return res.status(400).json({ message: 'Version ID is required for rollback.'
});
    const evidence = await Evidence.findById(req.params.id);
    if (!evidence || evidence.isDeleted) return res.status(404).json({ message: 'Evidence not
found.' });
    const targetVersion = await EvidenceVersion.findOne({ _id: versionId, evidenceId: evidence._id
});
    if (!targetVersion) return res.status(404).json({ message: 'Version record not found.' });
    const currentVersionCount = await EvidenceVersion.countDocuments({ evidenceId: evidence._id });
    const nextVersionNumber = currentVersionCount + 1;
    await EvidenceVersion.create({
      evidenceId: evidence._id, version: nextVersionNumber, updatedBy: req.user._id,
      changes: `Rollback to version ${targetVersion.version}`, encryptedFileURL:
evidence.encryptedFileURL,
      fileSize: evidence.fileSize, hash: evidence.hash, fileKeyEncrypted: evidence.fileKeyEncrypted,
fileIvHex: evidence.fileIvHex,
    });
    evidence.encryptedFileURL = targetVersion.encryptedFileURL; evidence.fileSize =
targetVersion.fileSize;
    evidence.hash = targetVersion.hash; evidence.fileKeyEncrypted = targetVersion.fileKeyEncrypted;
evidence.fileIvHex = targetVersion.fileIvHex;
    await evidence.save();
    await logAudit(req, { action: 'ROLLBACK', target: evidence._id.toString(), details: `Rollback to
v${targetVersion.version} logged.` });
    res.json({ message: `Successfully rolled back to version ${targetVersion.version}.`, evidence
});
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

module.exports = { getEvidenceList, getEvidenceById, uploadEvidence, downloadEvidence,
rollbackEvidenceVersion };
```

Deployment URL: http://localhost:5002 (Backend API) | http://localhost:5174 (Web Dashboard)

This controller forms the operational foundation of the CivicEvidence DEMS platform. By decoupling cryptographic operations into specialized services and enforcing immutable audit logging on every request context, it ensures strict compliance with ISO/IEC 27037 and NIST SP 800-86 standards.

APPENDIX B – INTERNSHIP CERTIFICATE

cloudstrand

INTERNSHIP COMPLETION CERTIFICATE

Dear Saravanan A *[CS-INT-2026-SA],*

On behalf of CloudStrand, we congratulate you on the successful completion of your four-week summer internship,. Your dedication and readiness to learn made a clear contribution to the team.

You worked as a Software Engineer Intern under the supervision of Mr. Pon Prabhakar - *Senior delivery manager - CloudStrand*

WORKED ON THESE AREAS

- During the internship, the intern contributed to developing a secure forensic investigation platform, covering backend development, data security, access control, and audit logging. The intern also built an interactive dashboard and integrated AI-powered features to improve evidence search and analysis.

We trust the placement has given you a sound foundation in cloud-ready software engineering, practical security, and professional development practice. We hope you will keep in touch and consider future opportunities with CloudStrand.

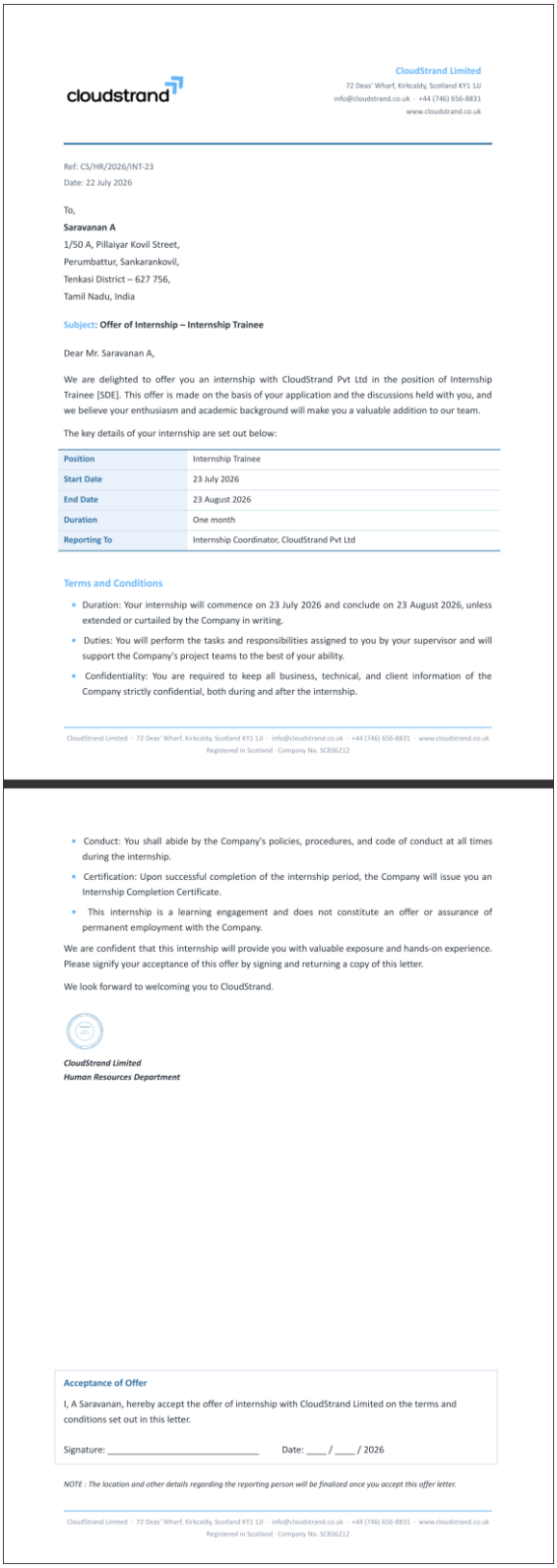
With thanks,

Mr. Pon Prabhakar

Internship Supervisor
CloudStrand Private Limited
Issued on 7th Oct 2026

www.cloudstrand.co.uk · CloudStrand Private Limited · Certificate CS-INT-2026-SA

APPENDIX C – OFFER / JOINING LETTER



cloudstrand

CloudStrand Limited
72 Deas' Wharf, Kirkcaldy, Scotland KY1 1JJ
info@cloudstrand.co.uk · +44 (746) 656-8831
www.cloudstrand.co.uk

Ref: CS/HR/2026/INT-23
Date: 22 July 2026

To,
Saravanan A
1/50 A, Pillaiyar Kovil Street,
Perumbattur, Sankarankovil,
Tenkasi District – 627 756,
Tamil Nadu, India

Subject: Offer of Internship – Internship Trainee

Dear Mr. Saravanan A,

We are delighted to offer you an internship with CloudStrand Pvt Ltd in the position of Internship Trainee [SDE]. This offer is made on the basis of your application and the discussions held with you, and we believe your enthusiasm and academic background will make you a valuable addition to our team.

The key details of your internship are set out below:

Position	Internship Trainee
Start Date	23 July 2026
End Date	23 August 2026
Duration	One month
Reporting To	Internship Coordinator, CloudStrand Pvt Ltd

Terms and Conditions

- Duration: Your internship will commence on 23 July 2026 and conclude on 23 August 2026, unless extended or curtailed by the Company in writing.
- Duties: You will perform the tasks and responsibilities assigned to you by your supervisor and will support the Company's project teams to the best of your ability.
- Confidentiality: You are required to keep all business, technical, and client information of the Company strictly confidential, both during and after the internship.

CloudStrand Limited · 72 Deas' Wharf, Kirkcaldy, Scotland KY1 1JJ · info@cloudstrand.co.uk · +44 (746) 656-8831 · www.cloudstrand.co.uk
Registered in Scotland · Company No. SC836212

- Conduct: You shall abide by the Company's policies, procedures, and code of conduct at all times during the internship.
- Certification: Upon successful completion of the internship period, the Company will issue you an Internship Completion Certificate.
- This internship is a learning engagement and does not constitute an offer or assurance of permanent employment with the Company.

We are confident that this internship will provide you with valuable exposure and hands-on experience. Please signify your acceptance of this offer by signing and returning a copy of this letter.

We look forward to welcoming you to CloudStrand.

CloudStrand Limited
Human Resources Department

Acceptance of Offer

I, A Saravanan, hereby accept the offer of internship with CloudStrand Limited on the terms and conditions set out in this letter.

Signature: _____ Date: ____ / ____ / 2026

NOTE : The location and other details regarding the reporting person will be finalized once you accept this offer letter.

CloudStrand Limited · 72 Deas' Wharf, Kirkcaldy, Scotland KY1 1JJ · info@cloudstrand.co.uk · +44 (746) 656-8831 · www.cloudstrand.co.uk
Registered in Scotland · Company No. SC836212

APPENDIX D – INTERNSHIP PRESENTATION EVALUATION SCORE SHEET

S.No.	Evaluation Criterion	CO	Max. Marks	Marks Awarded
1	Industry Exposure & Domain Understanding	CO1	20	
2	Problem Identification & Analysis	CO2	15	
3	Solution Design & Approach	CO2	15	
4	Implementation & Technical Skills	CO3	15	
5	Testing, Results & Deployment	CO3	10	
6	Technical Report & Documentation	CO4	10	
7	Presentation & Communication Skills	CO4	10	
8	Viva / Response to Questions	CO1–CO4	5	
	TOTAL		100	

Overall Marks _____ / 100

Evaluator Name & Signature: _____

Date: _____